

LAPORAN PRAKTIKUM
MATERI PRAKTIKUM : PROSES CONCEPT, PROSES
OPERATIONS, IPC



Dosen Pengampu :
Triawan Adi Cahyanto M.Kom

Disusun oleh :
MUHAMMAD RIDHO 2410651009 (A)

Tanggal : 10 – 21 - 2025

PRODI TEKNIK INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS MUHAMMADIYAH JEMBER
2025

1. Tujuan

1. Memahami konsep dasar proses dan siklus hidup proses dalam sistem operasi.
2. Mampu membuat dan mengelola proses menggunakan system call seperti `fork()`, `exec()`, dan `wait()`.
3. Memahami hubungan antara proses induk (parent) dan proses anak (child).
4. Mempelajari komunikasi antar proses (Interprocess Communication / IPC).
5. Mengimplementasikan berbagai metode IPC seperti **Pipe**, **FIFO (Named Pipe)**, dan **Shared Memory**.
6. Membandingkan performa berbagai metode komunikasi proses.

2. Dasar Teori

2.1 Konsep Proses

- **Proses** adalah program yang sedang dieksekusi. Setiap proses memiliki **Process ID (PID)** dan informasi terkait pada **Process Control Block (PCB)**.
- Dalam sistem operasi multitasking, banyak proses dapat berjalan secara bersamaan. Kernel mengatur penjadwalan CPU agar tiap proses mendapat waktu eksekusi yang adil.
- Pembuatan proses baru dilakukan dengan **system call** `fork()` yang menduplikasi proses induk menjadi anak.
- Eksekusi program lain dapat dilakukan menggunakan `exec()` sedangkan `wait()` digunakan agar parent menunggu anak selesai sebelum melanjutkan.

2.2 Konsep IPC (Interprocess Communication)

- **IPC** adalah mekanisme yang memungkinkan proses-proses saling bertukar data dan sinkronisasi.

- Tiga metode utama IPC:
 1. **Pipe (Anonymous Pipe)** – komunikasi satu arah antar proses yang memiliki hubungan parent-child.
 2. **FIFO (Named Pipe)** – memungkinkan komunikasi dua arah bahkan antar proses yang tidak berkerabat.
 3. **Shared Memory** – dua atau lebih proses mengakses area memori bersama untuk bertukar data dengan cepat.

• 3. Percobaan

• Percobaan 1.1 – Membuat Proses Sederhana

• Source

Code:

Program sederhana menggunakan fork() untuk menampilkan dua proses (parent dan child).

Analisis:

fork() mengembalikan 0 pada proses anak dan nilai PID anak pada proses induk. Terjadi duplikasi proses dari satu program menjadi dua eksekusi paralel.

Percobaan 1.1: Membuat Proses Sederhana

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>

int main() {
    pid_t pid;

    printf("Program dimulai\n");
    printf("PID proses ini: %d\n", getpid());
    printf("PID parent: %d\n", getppid());

    return 0;
}
```

```
root@RIDHO:~# nano process_basic.c
root@RIDHO:~# gcc -o process_basic process_basic.c
root@RIDHO:~# ./process_basic
Program dimulai
PID proses ini: 48
PID parent: 15
root@RIDHO:~# |
```

Percobaan 1.2: Fork - Parent dan Child Process

Percobaan 1.2 – Fork: Parent dan Child Process

Source

Code:

Program menampilkan perbedaan PID antara parent dan child.

Analisis:

Parent dan child menjalankan bagian kode yang sama tetapi memiliki ruang memori terpisah. Parent dapat mencetak PID anak, sedangkan child mencetak PID dirinya sendiri.

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>

int main() {
    pid_t pid;

    printf("Sebelum fork()\n");
    printf("PID: %d\n\n", getpid());

    pid = fork();

    if (pid < 0) {
        fprintf(stderr, "Fork gagal!\n");
        return 1;
    }

    else if (pid == 0) {
        //Child process
        printf("CHILD PROCESS\n");
        printf("PID saya: %d\n", getpid());
        printf("PID child saya: %d\n", pid);
    }
}
```

```
else {  
  
    //Parent process  
    printf("PARENT PROCESS\n");  
    printf("PID saya: %d\n", getpid());  
    printf("PID child saya: %d\n", pid);  
  
}  
  
printf("Proses %d selesai\n\n", getpid());  
return 0;  
}
```

```
root@RIDHO:~# nano fork_demo.c  
root@RIDHO:~# gcc -o fork_demo fork_demo.c  
root@RIDHO:~# ./fork_demo  
Sebelum fork()  
PID: 57  
  
PARENT PROCESS  
PID saya: 57  
PID child saya: 58  
Proses 57 selesai  
  
CHILD PROCESS  
PID saya: 58  
PID child saya: 0  
Proses 58 selesai
```

Percobaan 1.3: Process Termination dengan wait()

Percobaan 1.3 – Process Termination dengan wait()

SourceCode:

Parent membuat child lalu menunggu menggunakan wait().

Analisis:

wait() mencegah zombie process dengan memastikan parent menunggu sampai child selesai. Ini juga menunjukkan sinkronisasi antar proses.

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main() {
    pid_t pid;
    int status;

    pid = fork();

    if (pid == 0) {
        //Child process
        printf("Child: PID saya %d, akan tidur 3 detik\n", getpid());
        sleep(3);
        printf("Child: Selesai!\n");
        return 42; // Exit code
    }
    else {
        // Parent process
        printf("Parent: Menunggu child PID: %d selesai...\n", pid);
        wait(&status);

        if (WIFEXITED(status)) {
            printf("Parent: Child selesai dengan exit code: %d\n",
                WEXITSTATUS(status));
        }
    }
    return 0;
}
```

```
root@RIDHO:~# nano process_wait.c
root@RIDHO:~# gcc -o process_wait process_wait.c
root@RIDHO:~# ./process_wait
Child: PID saya 68, akan tidur 3 detik
Parent: Menunggu child PID: 68) selesai...
Child: Selesai!
Parent: Child selesai dengan exit code: 42
root@RIDHO:~# |
```

Percobaan 1.4: Multiple Child Processes

Percobaan 1.4 – Multiple Child Processes

SourceCode:

Program membuat beberapa child menggunakan loop dan fork().

Analisis:

Sistem operasi membedakan setiap child berdasarkan PID. Penggunaan wait() dalam loop mencegah proses zombie dan memastikan semua anak selesai.

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    int i;
    pid_t pid;
    int num_children = 3;

    printf("Parent PID: %d\n\n", getpid());

    for (i = 0; i < num_children; i++) {
        pid = fork();

        if (pid == 0) {
            //Child process
            printf("Child %d selesai\n", i+1);
            return i;
        }
    }

    //Parent menunggu semua child
    for (i = 0; i < num_children; i++) {
        int status;
        pid_t child_pid = wait(&status);
        printf("Parent: Child dengan PID %d selesai\n", child_pid);
    }

    printf("\nParent: Semua child selesai\n");

    return 0;
}
```

```
root@RIDHO:~# nano multiple_fork.c
root@RIDHO:~# gcc -o multiple_fork multiple_fork.c
root@RIDHO:~# ./multiple_fork
Parent PID: 77

Child 1 selesai
Child 2 selesai
Parent: Child dengan PID 78 selesai
Parent: Child dengan PID 79 selesai
Child 3 selesai
Parent: Child dengan PID 80 selesai

Parent: Semua child selesai
```


Percobaan 2.1: Pipes - Communication One Way

Percobaan 2.1 – Pipes: One Way Communication

SourceCode:

Parent menulis ke pipe, child membaca dari pipe.

Analisis:

Pipe berfungsi sebagai buffer sementara. Komunikasi bersifat satu arah — data mengalir dari parent ke child atau sebaliknya. Cocok untuk komunikasi cepat antar proses yang saling terkait.

```
#include <stdio.h>
#include <unistd.h>
#include <string.h>
#include <stdlib.h>
#include <sys/wait.h>

int main() {
int pipefd[2];
pid_t pid;
char write_msg[] = "Hello dari Parent!";
char read_msg[100];

//Buat pipe
if (pipe(pipefd) == -1) {
perror("Pipe gagal");
return 1;
}

pid = fork();

if (pid == -1) {
perror("Fork gagal");
return 1;
}
```

```
if (pid == 0) {
//Child process - membaca dari pipe
close(pipefd[1]); //Tutup write end
read(pipefd[0], read_msg, sizeof(read_msg));
printf("Child menerima: %s\n", read_msg);
close(pipefd[0]);
} else {
//Parent process - menulis ke pipe
close(pipefd[0]);
printf("Parent mengirim: %s/n", write_msg);
write(pipefd[1], write_msg, strlen(write_msg) +1);
close(pipefd[1]);
wait(NULL);
}
return 0;
}
```

```
root@RIDHO:~# nano pipe_mode.c
root@RIDHO:~# gcc -o pipe_mode pipe_mode.c
root@RIDHO:~# ./pipe_mode
Child menerima: Hello dari Parent!
Parent mengirim: Hello dari Parent!/nroot@RIDHO:~#
```

Percobaan 2.2: Named Pipes (FIFO) - Two Way Communication

Percobaan 2.2 – Named Pipe (FIFO): Two Way Communication

SourceCode:

Dua program berbeda saling berkomunikasi menggunakan FIFO (mkfifo).

Analisis:

FIFO dapat diakses oleh proses yang tidak memiliki hubungan parent-child. Komunikasi bisa dua arah, sehingga cocok untuk sistem client-server sederhana.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <sys/stat.h>
#include <unistd.h>

#define FIFO_FILE "/tmp/my_fifo"

int main() {
    int fd;
    char buffer[100];

    //Buat FIFO jika belum ada
    mkfifo(FIFO_FILE, 0666);

    printf("Writer: Menunggu reader...\n");
    fd = open(FIFO_FILE, O_WRONLY);

    printf("Masukkan pesan: ");
    fgets(buffer, sizeof(buffer), stdin);

    write(fd, buffer, strlen(buffer) + 1);
    printf("Writer: Pesan terkirim!\n");

    close(fd);

    return 0;
}
```

```
root@RIDHO:~# gcc -o fifo_writer fifo_writer.c
root@RIDHO:~# ./fifo_writer
Writer: Menunggu reader...
Masukkan pesan: HALO SAYA MUHAMMAD RIDHO
Writer: Pesan terkirim!
root@RIDHO:~# |
```

```

#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <sys/stat.h>
#include <unistd.h>

#define FIFO_FILE "/tmp/my_fifo"
int main() {
    int fd;
    char buffer[100];

    mkfifo(FIFO_FILE, 0666);

    printf("Reader: Membuka FIFO...\n");
    fd = open(FIFO_FILE, O_RDONLY);

    read(fd, buffer, sizeof(buffer));
    printf("Reader: Menerima pesan: %s\n", buffer);

    close(fd);
    unlink(FIFO_FILE); //Hapus FIFO

    return 0;
}

```

```

root@RIDHO:~# gcc -o fifo_reader fifo_reader.c
root@RIDHO:~# ./fifo_reader
Reader: Membuka FIFO...
Reader: Menerima pesan: HALO SAYA MUHAMMAD RIDHO

```

Percobaan 2.3: Shared Memory IPC

Percobaan 2.3 – Shared Memory IPC

SourceCode:

Implementasi menggunakan shmget(), shmat(), shmdt(), dan shmctl() untuk membuat memori bersama.

Analisis:

Shared memory memungkinkan proses membaca dan menulis langsung ke area memori yang sama. Kecepatannya tinggi karena tidak ada perantara, tetapi butuh mekanisme sinkronisasi seperti semaphore agar tidak terjadi konflik data.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/ipc.h>
#include <sys/shm.h>

#define SHM_SIZE 1024

int main() {
    key_t key;
    int shmid;
    char *shared_memory;
    char message[100];

    //Generate unique key
    key = ftok("shmfile", 65);

    //Create shared memory
    shmid = shmget(key, SHM_SIZE, 0666 | IPC_CREAT);

    //Attach to shared memory
    shared_memory = (char*) shmat(shmid, NULL, 0);

    printf("Writer: Masukkan pesan: ");
    fgets(message, sizeof(message), stdin);

    //Write to shared memory
    strcpy(shared_memory, message);
    printf("Writer: Data ditulis ke shared memory\n");

    //Deteach from shared memory
    shmdt(shared_memory);
    return 0;
}
```

```
root@RIDHO:~# gcc -o shm_writer shm_writer.c
root@RIDHO:~# ./shm_writer
Writer: Masukkan pesan: HALO MUHAMMAD RIDHO
Writer: Data ditulis ke shared memory
root@RIDHO:~# |
```

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/ipc.h>
#include <sys/shm.h>

#define SHM_SIZE 1024

int main() {
    key_t key;
    int shmid;
    char *shared_memory;

    //Generate same key
    key = ftok("shmfile", 65);

    //Acces shared memory
    shmid = shmget(key, SHM_SIZE, 0666);

    //Attach to shared memory
    shared_memory = (char*) shmat(shmid, NULL, 0);

    //Detach and remove shared memory
    shmdt(shared_memory);
    shmctl(shmid, IPC_RMID, NULL);

    return 0;
}
```

```
root@RIDH0:~# gcc -o shm_reader shm_reader.c
root@RIDH0:~# ./shm_reader
root@RIDH0:~# |
```

Percobaan 2.4: Perbandingan Performa IPC

Percobaan 2.4 – Perbandingan Performa IPC

Source Code:

Program mengukur waktu kirim data antar proses menggunakan pipe dan shared memory.

Analisis:

Hasil menunjukkan **Shared Memory** lebih cepat dibandingkan **Pipe** atau **FIFO**, karena transfer data tidak perlu melewati kernel buffer.

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/time.h>
#include <sys/wait.h>
#include <string.h>
#include <unistd.h>
#include <sys/ipc.h>
#include <sys/shm.h>

#define DATA_SIZE 1000000
#define SHM_SIZE DATA_SIZE * sizeof(int)

double get_time() {
    struct timeval tv;
    gettimeofday(&tv, NULL);
    return tv.tv_sec + tv.tv_usec / 1000000.0;
}

void test_pipe() {
    int pipefd[2];
    pid_t pid;
    int data[DATA_SIZE];
    double start, end;

    pipe(pipefd);
    start = get_time();

    pid = fork();

    if(pid == 0) {
        close(pipefd[1]);
        read(pipefd[0], data, sizeof(data));
        close(pipefd[0]);
        exit(0);
    }
```

```
else {
close(pipefd[0]);

for (int i = 0; i < DATA_SIZE; i++) {
data[i] = i;
}

write(pipefd[1], data, sizeof(data));
close(pipefd[1]);
wait(NULL);

end = get_time();
printf("Pipe: %.6f detik\n", end - start);
}
}

void test_shared_memory() {
key_t key = ftok("shmfile", 65);
int shmid;
int *shared_data;
pid_t pid;
double start, end;

shmid = shmget(key, SHM_SIZE, 0666 | IPC_CREAT);
start = get_time();

pid = fork();
```



```

if(pid == 0) {
shared_data = (int*) shmat(shmid, NULL, 0);
//Child hanya membaca
int sum = 0;
for (int i = 0; i < 100; i++) {
sum += shared_data[i];
}
shmdt(shared_data);
exit(0);
}

else {
shared_data = (int*) shmat(shmid, NULL, 0);

for(int i = 0; i < DATA_SIZE; i++) {
shared_data[i] = i;
}

wait(NULL);
end = get_time();

printf("Shared Memory: %.6f detik\n", end - start);

shmdt(shared_data);
shmctl(shmid, IPC_RMID, NULL);
}
}

int main() {
printf("Benchmarking IPC Methods dengan %d integers\n\n", DATA_SIZE);

test_pipe();
test_shared_memory();

return 0;
}

```

```

root@RIDHO:~# nano ipc_benchmark.c

root@RIDHO:~# gcc -o ipc_benchmark ipc_benchmark.c
root@RIDHO:~# ./ipc_benchmark
Benchmarking IPC Methods dengan 1000000 integers

root@RIDHO:~# |

```

4. Tugas Akhir

Implementasi Sistem IPC dengan Shared Memory dan FIFO

Penjelasan Algoritma:

1. Proses induk membuat shared memory menggunakan `shmget()`.
2. Proses anak terhubung menggunakan `shmat()`.
3. Proses induk menulis data, anak membaca data dan menampilkannya.
4. Sinkronisasi dilakukan dengan flag sederhana atau semaphore.
5. Untuk FIFO, dua file FIFO dibuat untuk komunikasi dua arah antara dua proses yang berbeda.

Analisis Performa:

- Shared memory unggul dalam kecepatan transfer.
- FIFO unggul dalam fleksibilitas karena tidak butuh hubungan parent-child.
- Kombinasi keduanya bisa digunakan untuk sistem besar (misalnya client-server).

5. Kesimpulan

1. Proses adalah unit terkecil eksekusi dalam sistem operasi yang dapat berjalan bersamaan (multitasking).
2. System call seperti `fork()`, `exec()`, dan `wait()` digunakan untuk membuat, menjalankan, dan mengelola proses.
3. Komunikasi antar proses (IPC) penting untuk koordinasi dan pertukaran data antar program.
4. Pipe dan FIFO lebih sederhana, tetapi shared memory lebih cepat dan efisien untuk transfer data besar.
5. Melalui percobaan ini, mahasiswa memahami hubungan parent-child process, mekanisme IPC, serta dapat menganalisis performa antar metode komunikasi.

6. Pembelajaran

Dari praktikum ini, mahasiswa belajar:

- Bagaimana sistem operasi menangani proses, komunikasi, dan sinkronisasi.
- Cara menerapkan system call pada C untuk mengelola proses dan komunikasi.
- Mengukur serta membandingkan performa berbagai metode IPC.